

인턴 온보딩 · 기술 참고자료

웹 서버와 WAS 완전 정복

브라우저에서 **Enter**를 누른 순간부터 화면이 뜨기까지,
요청 한 번에 벌어지는 모든 일

클라이언트-서버

Apache · Nginx

Tomcat · WildFly

Spring Boot

대상 · 초급 개발자 / 웹 개발 입문 인턴 | 예상 소요 · 약 30분 | 선수 지식 · 스프링부트 교육 수강 권장

이 문서에서 배우는 것

1. 큰 그림 — 클라이언트와 서버 요청과 응답이라는 웹의 기본 대화
2. 요청은 어떤 길로 서버까지 가나 주소창 Enter → DNS → TCP → HTTP → 응답의 전 과정
3. 웹 서버 — Apache와 Nginx 요청을 가장 먼저 받는 최전선
4. WAS — Tomcat과 WildFly 프로그램(비즈니스 로직)이 실제로 도는 곳
5. 웹 서버 vs WAS — 무엇이 다른가 왜 둘을 같이 쓰나
6. Spring Boot와 서버 — 내장 톰캣의 정체 여러분이 매일 쓰던 그것
7. 알면 좋은 점 — 개발자 관점의 실익 에러 진단·배포·면접까지
8. 한 장 요약 · 용어사전 · 더 보기

먼저 결론부터 (한 문단 요약)

여러분이 브라우저 주소창에 URL을 치면, 요청은 **DNS로 주소를 찾고 → TCP로 연결하고 → HTTP로 내용을 실어** 서버에 도착합니다. 서버 앞단에는 요청을 가장 먼저 받는 **웹 서버(Nginx·Apache)**가 있고, 그 뒤에 실제 프로그램이 도는 **WAS(Tomcat·WildFly)**가 있습니다. 웹 서버는 **정적인 파일과 교통정리**를, WAS는 **동적인 계산과 DB 처리**를 맡습니다. 스프링부트는 이 WAS(톰캣)를 **프로그램 안에 내장해** 버튼 하나로 서버를 띄워 줍니다. 이 구조를 알면 “왜 8080인가”, “502 에러는 누구 탓인가”, “어떻게 배포하나”가 한눈에 보입니다.

1 큰 그림 — 클라이언트와 서버

서버(Server)는 말 그대로 “무언가를 제공(serve)하는 쪽”, 클라이언트(Client)는 “요청하는 쪽”입니다. 웹의 모든 동작은 이 둘 사이의 **요청(Request)**과 **응답(Response)**이라는 짧은 대화의 반복입니다.

☕ 카페 비유로 시작하기

여러분(클라이언트)이 카운터에서 “아메리카노 주세요”라고 **주문(요청)**합니다. 직원(서버)이 커피를 만들어 **내어줍니다(응답)**. 이 문서에 나오는 모든 등장인물 — 웹 서버, WAS, DB — 은 이 카페 주방 안의 서로 다른 담당자라고 생각하면 됩니다. 손님은 주방 구조를 몰라도 커피만 받으면 되지만, **주방을 아는 직원(개발자)**은 주문이 밀릴 때 어디가 병목인지 압니다.



그림 1. 웹의 기본 대화 — 요청과 응답

중요한 점: **한 대의 컴퓨터가 상황에 따라 클라이언트도, 서버도 될 수 있습니다.** “서버”는 특별한 하드웨어가 아니라 **요청을 기다렸다가 응답해 주는 프로그램(역할)**입니다. 여러분의 노트북에서 스프링부트를 실행하면, 그 순간 노트북이 서버가 됩니다.

2 요청은 어떤 길로 서버까지 가나

주소창에 `https://www.example.com/intro/1` 을 치고 **Enter**를 누르면, 화면이 뜨기까지 눈에 보이지 않는 여러 단계가 순식간에 일어납니다. 하나씩 따라가 봅시다.

- ① **URL 해석**
주소를 프로토콜(https) · 도메인(example.com) · 경로(/intro/1)로 분해
- ② **DNS 조회**
도메인 이름 → IP 주소로 변환 (예: example.com → 93.184.x.x)
- ③ **TCP 연결**
IP:포트(443)로 3-way handshake — “연결할까요?/좋아요/시작!”
- ④ **TLS 암호화 (HTTPS일 때)**
서로 인증서를 확인하고 암호 키를 교환 → 이후 통신은 암호화
- ⑤ **HTTP 요청 전송**
“GET /intro/1 주세요” — 메서드 · 경로 · 헤더를 담아 서버로
- ⑥ **서버 처리 & HTTP 응답**
웹서버→WAS→(DB)가 처리 → 상태코드(200) + HTML을 응답으로 반환
- ⑦ **브라우저 렌더링**
받은 HTML/CSS/JS를 해석해 화면을 그림

그림 2. 주소창 Enter → 화면 표시까지의 전체 경로

2-1. IP 주소와 DNS — 인터넷의 전화번호부

모든 서버는 **IP 주소**(예: `93.184.216.34`)라는 숫자 주소를 가집니다. 하지만 사람이 숫자를 외우긴 어렵죠. 그래서 `example.com` 같은 **도메인 이름**을 쓰고, 이를 IP로 바꿔 주는 것이 **DNS(Domain Name System)**입니다.

▣ 비유

DNS는 **전화번호부**와 같습니다. “김서방”(도메인)이라는 이름으로 “010-1234-5678”(IP)을 찾아 주는 것이죠. 전화를 걸려면 결국 **번호(IP)**가 필요하듯, 통신도 결국 IP로 이뤄집니다.

2-2. 포트(Port) — 한 건물의 여러 사무실

IP 주소가 **건물 주소**라면, **포트 번호**는 그 건물 안의 **사무실 호수**입니다. 한 서버(IP)에서 여러 프로그램이 각자 다른 포트로 손님을 받습니다.

포트	용도	비고
80	HTTP (암호화 안 됨)	웹의 기본 포트. 생략 가능
443	HTTPS (암호화)	오늘날 웹의 표준
8080	Tomcat 등 WAS 기본 포트	개발 중 localhost:8080 이 바로 이것
3306 / 5432	MySQL / PostgreSQL	DB도 포트로 접속

💡 여기서 첫 번째 “아하!”

스프링부트를 실행하면 콘솔에 `Tomcat started on port 8080` 이 났던 것 기억나나요? 그건 **여러분의 WAS(툼캣)가 8080번 사무실 문을 열고 손님을 기다린다는** 뜻이었습니다. 그래서 브라우저에 `localhost:8080` 으로 접속했던 것이죠. (localhost = “내 컴퓨터”)

2-3. TCP 연결과 3-way handshake

데이터를 주고받기 전에, 클라이언트와 서버는 먼저 **연결(통로)을 확립**합니다. 이때 세 번의 인사를 주고받는데 이를 **3-way handshake**라고 합니다.

1. **SYN** — 클라이언트: “연결하고 싶어요” (통화 연결음)
2. **SYN-ACK** — 서버: “좋아요, 저도 준비됐어요”
3. **ACK** — 클라이언트: “그럼 시작합니다” → 연결 성립

TCP는 데이터가 **순서대로, 빠짐없이** 도착하도록 보장합니다. (웹·이메일 등 정확성이 중요한 통신의 기반)

2-4. HTTP — 실제 대화의 언어

연결이 되면, 그 위에서 **HTTP(HyperText Transfer Protocol)**라는 약속된 형식으로 실제 내용을 주고받습니다.

```
# 클라이언트가 보내는 HTTP 요청
GET /intro/1 HTTP/1.1      ← 메서드 · 경로 · 버전
Host: www.example.com
Accept: text/html
Cookie: sessionId=abc123   ← 헤더(부가 정보)
# (본문은 GET에선 보통 비어 있음)
```

```
# 서버가 돌려주는 HTTP 응답
HTTP/1.1 200 OK           ← 상태코드
Content-Type: text/html
Content-Length: 1024

<html>...자기소개서 상세 화면...</html> ← 본문(실제 내용)
```

자주 쓰는 HTTP 메서드

메서드	의미	예
GET	조회 (데이터를 달라)	목록/상세 보기
POST	생성 (새로 만들어라)	글 등록
PUT / PATCH	수정	글 수정
DELETE	삭제	글 삭제

상태 코드(Status Code) — 응답의 결과 요약

범위	의미	대표 예
2xx	성공	200 OK, 201 Created
3xx	리다이렉트(다른 곳으로)	301 영구 이동, 302 임시
4xx	클라이언트 잘못	404 없음, 401/403 인증·권한
5xx	서버 잘못	500 내부 오류, 502 게이트웨이

💡 실무 감각

4xx는 “요청한 쪽 문제”, 5xx는 “서버 쪽 문제”라고 외워두세요. 에러를 만났을 때 누구를 먼저 의심할지가 바로 갈립니다. (예: 404 는 URL을 잘못 친 것, 500 은 서버 코드에 예외가 터진 것)

3 웹 서버 — Apache와 Nginx

이제 요청이 서버 건물에 도착했습니다. 그 건물의 정문에서 손님을 가장 먼저 맞이하는 안내데스크가 바로 웹 서버(Web Server)입니다. 대표 주자가 Apache HTTP Server와 Nginx입니다.

3-1. 웹 서버가 하는 일

- 정적 콘텐츠 제공 — 이미 만들어져 있어 그대로 내보내면 되는 파일(HTML, CSS, JS, 이미지)을 빠르게 전달
- 요청 수신 · 교통정리 — 들어온 요청을 받아서, 필요하면 뒤쪽 WAS로 넘김 (다음 절의 리버스 프록시)
- 부가 기능 — HTTPS 암호화 처리, 접속 로그, 접근 제어, 캐싱, 압축 등

✦ 정적 콘텐츠 vs 동적 콘텐츠

정적(static) = 누가 언제 요청해도 똑같은 결과 (회사 로고 이미지, 고정된 안내 페이지). 동적(dynamic) = 요청·사용자·시점에 따라 매번 달라지는 결과 (내 자기소개서 목록, 로그인한 사용자 이름). 웹 서버는 정적을, WAS는 동적을 잘합니다. 이 한 줄이 3~5장의 핵심입니다.

3-2. Apache HTTP Server

1995년부터 이어진 가장 오래되고 검증된 웹 서버입니다. (오픈소스, Apache 재단)

- 모듈 방식으로 기능을 붙였다 뗄 수 있어 유연합니다.
- `.htaccess` 파일로 디렉터리별 세밀한 설정이 가능해 공유 호스팅에서 특히 많이 쓰였습니다.
- 전통적으로 요청 하나당 프로세스/스레드 하나를 배정하는 방식(prefork)이라, 접속이 폭증하면 메모리 부담이 큼니다. (지금은 이벤트 기반 방식도 지원하지만, 이 "요청당 스레드" 이미지가 Nginx와의 대비점입니다.)

3-3. Nginx (엔진엑스)

2004년 등장, 폭증하는 동시 접속(이른바 C10K 문제)을 겨냥해 만들어진 고성능 웹 서버입니다.

- 이벤트 기반 · 비동기 구조라 적은 자원으로 많은 동시 접속을 처리합니다.
- 정적 파일 전송과 리버스 프록시 · 로드 밸런싱이 특히 강력해, 오늘날 신규 서비스의 기본 선택이 되었습니다.
- 설정이 간결합니다.

3-4. 리버스 프록시(Reverse Proxy) — 이 문서의 숨은 주인공

리버스 프록시는 "손님 대신 받아서, 뒤쪽 실무자에게 넘겨 주는 대리 창구"입니다. Nginx/Apache가 요청을 먼저 받아, 동적 처리가 필요하면 뒤에 있는 WAS(톰캣 등)로 전달하고, WAS의 응답을 다시 손님에게 돌려줍니다.

📖 비유 — 대형 병원의 접수처

환자(요청)가 오면 **접수처(웹 서버)**가 먼저 맞이합니다. 단순 서류 발급(정적 파일)은 접수처가 바로 처리하고, 진료가 필요하다면(동적 처리) **안쪽 진료실(WAS)**로 안내합니다. 환자는 진료실 위치를 몰라도 되고, 접수처가 교통정리·보안·순번(로드 밸런싱)을 담당합니다.

3-5. Apache vs Nginx 요약

비교 항목	Apache	Nginx
처리 방식	요청당 프로세스/스레드(전통)	이벤트 기반 · 비동기
동시 접속 대량 처리	상대적으로 무거움	가볍고 강함
정적 파일 전송	양호	매우 빠름
리버스 프록시/로드밸런싱	가능	강점 · 주력 용도
디렉터리별 설정	<code>.htaccess</code> 로 유연	중앙 설정(파일 단위)
대표 이미지	오래되고 안정적	현대적·고성능 표준

⚠️ 균형 잡힌 시각

“Nginx가 무조건 낫다”는 아닙니다. 기존 시스템·특정 모듈·`.htaccess` 의존 환경에서는 Apache가 여전히 적합합니다. 실제로 **둘을 함께 쓰기도** 합니다(예: Nginx가 앞단, Apache가 뒷단). 도구는 **상황에 맞게** 고르는 것입니다.

4 WAS — Tomcat과 WildFly

WAS(Web Application Server, 웹 애플리케이션 서버)는 실제 프로그램(애플리케이션)이 도는 곳입니다. 요청을 받아 자바 코드를 실행하고, DB에서 데이터를 꺼내 계산하고, 그 결과로 매번 다른(동적) 응답을 만들어 냅니다.

4-1. WAS가 하는 일

- 동적 콘텐츠 생성 — 로그인한 사용자에게 맞는 화면, DB 조회 결과 등
- 비즈니스 로직 실행 — 여러분이 작성한 Controller·Service 코드가 여기서 돌니다
- DB 연동, 트랜잭션, 세션 관리 등 애플리케이션 실행에 필요한 환경 제공
- 서블릿 컨테이너 역할 — 자바 웹의 표준 규격(Servlet)을 실행하는 엔진

☐ 서블릿(Servlet)이란?

자바로 웹 요청을 처리하는 표준 방식입니다. 스프링부트 교육에서 나온 `DispatcherServlet` 이 바로 그 서블릿의 하나예요. WAS = 서블릿을 실행해 주는 컨테이너(그릇)라고 이해하면 됩니다. 개발자는 로직만 짜고, 실행 환경은 WAS가 책임집니다.

4-2. Apache Tomcat (톰캣)

- 자바 서블릿/JSP를 실행하는 가장 대중적인 WAS. 가볍고 빠릅니다. (Apache 재단, 오픈소스)
- 정확히는 “서블릿 컨테이너”입니다 — 무거운 엔터프라이즈 규격 전부가 아니라, 웹 실행에 필요한 핵심 규격에 집중합니다.
- 스프링부트가 기본으로 내장하는 바로 그 서버 — 여러분이 이미 매일 쓰고 있었습니다(6장에서 자세히).

4-3. WildFly (와일드플라이, 구 JBoss)

- Red Hat이 주도하는 풀 스택 자바 애플리케이션 서버. 과거 이름은 JBoss AS입니다.
- 서블릿뿐 아니라 Jakarta EE(구 Java EE)의 풍부한 기능(EJB, JMS 메시징, 분산 트랜잭션 등)을 폭넓게 제공합니다.
- 대형·전통적 엔터프라이즈 시스템(대기업·공공·금융 등)에서 많이 쓰입니다. 그만큼 무겁고 설정이 복잡한 편입니다.

🔴 Tomcat vs WildFly, 한 줄 정리

Tomcat = “딱 웹 실행에 필요한 만큼”의 가벼운 서블릿 컨테이너 (대부분의 웹서비스·스프링부트에 충분).

WildFly = “자바 엔터프라이즈 기능을 다 갖춘” 풀 기능 WAS (대형 시스템에서 진가).

엄밀히는 “톰캣은 완전한 WAS가 아니라 서블릿 컨테이너”라는 시각도 있지만, 실무에서는 동적 처리를 하는 자바 서버라는 뜻으로 톰캣도 WAS로 묶어 부릅니다.

5 웹 서버 vs WAS — 무엇이 다른가

이제 핵심입니다. 둘의 **역할이 다르기 때문에**, 실무에서는 **웹 서버를 앞에, WAS를 뒤에** 놓고 **함께** 씁니다.

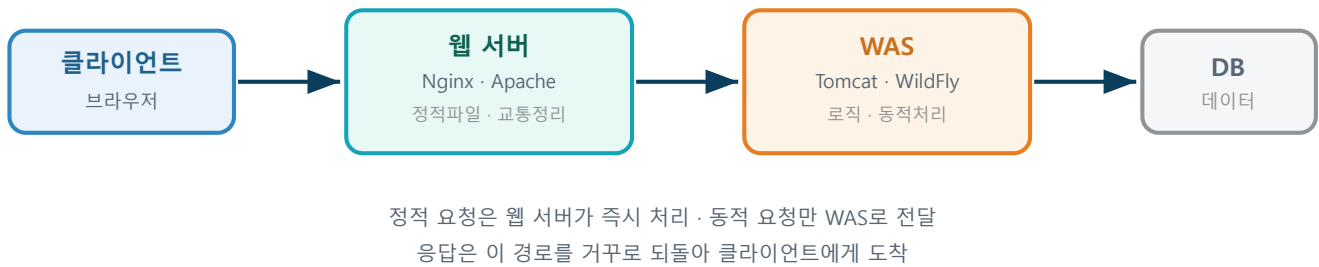


그림 3. 웹 서버 + WAS + DB의 협업 구조 (실무 표준)

5-1. 한눈에 보는 차이

비교 항목	웹 서버 (Nginx·Apache)	WAS (Tomcat·WildFly)
주 역할	정적 콘텐츠 제공, 요청 접수	동적 콘텐츠 생성, 로직 실행
대표 처리	이미지/HTML/CSS 파일 전송	DB 조회, 계산, 사용자별 화면
프로그램 실행	거의 안 함(전달자)	자바 코드가 실제로 돌
DB 연동	보통 안 함	함
위치	맨 앞(정문)	그 뒤(내부)
비유	병원 접수처	진료실

5-2. 왜 굳이 둘을 나눠 쓰나?

1. **효율** — 단순 정적 파일까지 무거운 WAS가 처리하면 낭비. 가벼운 웹 서버가 정적을 쳐내면 WAS는 **동적 처리에만 집중**.
2. **보안** — WAS를 외부에 직접 노출하지 않고, **웹 서버 뒤에 숨겨** 공격 표면을 줄임.
3. **확장(로드 밸런싱)** — 트래픽이 늘면 WAS를 **여러 대로 늘리고**, 앞단 웹 서버가 요청을 **고르게 분배**.
4. **HTTPS 집중 처리** — 암호화(SSL) 해제를 앞단 웹 서버가 전담(SSL 종료)하면 WAS는 부담을 덜어냄.

⚠ 경계는 점점 흐려진다 (오해 방지)

“웹 서버는 정적, WAS는 동적”은 **이해를 위한 큰 원칙**이지 절대 규칙이 아닙니다. Nginx도 간단한 동적 처리(스크립트·캐시)를 할 수 있고, WAS(톰캣)도 정적 파일을 직접 줄 수 있습니다. 또 소규모 서비스는 **WAS 하나만으로도** 충분히 운영됩니다(다음 장의 스프링부트가 그 예). “**원칙을 알되, 상황에 따라 유연하게**” — 이것이 실무의 태도입니다.

6 Spring Boot와 서버 — 내장 톰캣의 정체

여기서 지금까지의 내용이 여러분의 실습과 연결됩니다. 스프링부트 교육에서 `main()` 을 실행하니 바로 서버가 떴죠? 설치한 적도 없는 톰캣이 어떻게 떴을까요?

6-1. 스프링부트는 WAS(톰캣)를 “내장”한다

과거 자바 웹은 톰캣을 따로 설치하고, 완성한 프로그램을 `.war` 파일로 만들어 그 톰캣에 얹어(배포) 실행했습니다. 초기 설정이 무거웠죠.

스프링부트는 발상을 뒤집었습니다. 톰캣을 프로그램 안에 포함(내장)시켜, `.jar` 파일 하나로 만들고 `main()` 실행만으로 서버가 뜨게 했습니다.

```
// build.gradle - 이 한 줄에 "내장 톰캣"이 함께 들어옵니다
implementation 'org.springframework.boot:spring-boot-starter-web'
```

```
# 그래서 실행하면 콘솔에 이렇게 뜹니다
Tomcat started on port 8080 (http) ← 내장 톰캣이 8080에서 대기 시작
```

💡 지난 실습과 연결

스프링부트 교육 “02. 프로젝트 구조”에서 본 요청 흐름 — 내장 톰캣 → DispatcherServlet → Controller → Service → Repository → DB → 템플릿 → 응답 — 기억나나요? 그 맨 앞의 “내장 톰캣”이 바로 이 문서에서 말한 WAS입니다. 이제 그 정체를 정확히 알게 된 것이죠.

6-2. 그럼 Nginx·Apache는 언제 필요한가?

개발·소규모 단계에서는 내장 톰캣(WAS) 하나로 충분합니다. 하지만 실제 서비스로 배포할 때는 대개 앞단에 Nginx를 리버스 프록시로 둡니다.

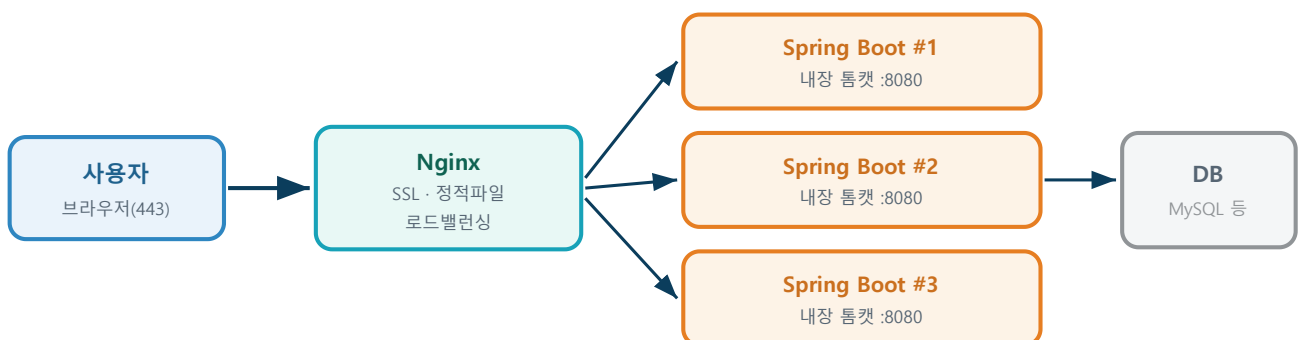


그림 4. 실무 배포 예시 — Nginx(앞단) + 여러 대의 Spring Boot(내장 톰캣) + DB

Nginx를 앞에 두면 이런 이점이 생깁니다:

- **HTTPS(SSL) 종료** — 인증서·암호화를 Nginx가 전담, 스프링부트는 순수 로직에 집중
- **정적 파일 오프로딩** — 이미지·CSS·JS는 Nginx가 빠르게 처리
- **로드 밸런싱** — 스프링부트 인스턴스 여러 대에 요청을 분산해 대용량 처리
- **보안 · 유연성** — WAS를 외부에 직접 노출하지 않고, 도메인/경로 라우팅을 앞단에서 제어

📦 내장형(jar) vs 외장형(war), 한 줄

내장형: 톰캣을 품은 `.jar` 를 그대로 실행(스프링부트 기본, 요즘 대세). **외장형:** `.war` 를 따로 설치한 톰캣/WildFly에 배포(전통 방식, 기존 시스템에서 여전히 사용). 참고로 스프링부트는 톰캣 대신 **Jetty·Undertow**(WildFly의 웹 엔진)로 교체할 수도 있습니다.

7 알면 좋은 점 — 개발자 관점의 실익

“스프링부트가 알아서 해 주는데 왜 이걸 알아야 하나요?” 좋은 질문입니다. 문제가 터졌을 때 그 차이가 드러납니다.

7-1. 에러가 “어느 층에서” 났는지 바로 안다

증상	의심 지점	해석
502 Bad Gateway	웹 서버 ↔ WAS 사이	Nginx는 살아 있는데 뒤의 스프링부트가 죽었거나 응답 없음
504 Gateway Timeout	WAS 처리 지연	WAS가 제한 시간 안에 응답을 못 줌(느린 쿼리 등)
404 Not Found	경로/라우팅	URL 매핑이 없음(웹서버 설정 or 컨트롤러 경로)
500 Internal Error	WAS 내부(내 코드)	스프링부트 코드에서 예외가 터짐 → 로그 확인
413 Payload Too Large	웹 서버 업로드 제한	Nginx의 <code>client_max_body_size</code> 에 걸림

💡 이것이 핵심 실익

구조를 모르면 502 를 보고 애먼 코드만 뒤집습니다. 구조를 알면 “아, Nginx는 멀쩡한데 톰캣이 안 떴구나 → 스프링부트 로그부터 보자”로 진단이 5분 만에 끝납니다. 계층을 아는 사람이 디버깅을 빨리 합니다.

7-2. 포트·주소가 더 이상 헷갈리지 않는다

`localhost:8080` (개발, 내장 톰캣)과 `https://우리서비스.com` (운영, 443의 Nginx 뒤)의 관계가 명확해집니다. “왜 로컬은 8080인데 배포하면 포트가 안 보이지?” 같은 질문이 자연스럽게 풀립니다(443/80은 생략되니까요).

7-3. 배포와 성능 튜닝의 감을 잡는다

- 스레드 풀 — 톰캣이 동시에 처리할 수 있는 요청 수(스레드)를 이해하면, 트래픽 급증 시 왜 느려지는지 보입니다.
- 정적 리소스 분리 — 이미지/CSS를 Nginx나 CDN으로 빼면 WAS 부담이 줄어듭니다.
- 수평 확장 — “서버를 늘린다”가 곧 “WAS 인스턴스를 늘리고 앞단에서 분산한다”임을 압니다.

7-4. 보안과 협업의 기본기가 된다

- HTTPS를 앞단에서 처리하는 이유, WAS를 외부에 직접 노출하면 안 되는 이유를 이해합니다.
- 인프라·운영(DevOps) 담당자와 같은 언어로 대화할 수 있습니다.

7-5. 면접·기술 대화에서 강해진다

📌 단골 질문 미리보기

- "웹 서버와 WAS의 차이는?" · "Apache와 Nginx의 차이는?" · "리버스 프록시가 뭔가요?"
- "스프링부트 내장 톰캣이란?" · "502와 500의 차이는?" — **전부 이 문서 한 편에 답이 있습니다.**

8 한 장 요약 · 용어사전 · 더 보기

핵심 8문장 요약

1. 웹의 기본

클라이언트가 요청, 서버가 응답. 그 반복이 전부다.

2. 통신 경로

URL→DNS(주소찾기)→TCP(연결)→(TLS)→HTTP(내용)→응답.

3. 포트

IP는 건물, 포트는 호수. 80=HTTP, 443=HTTPS, 8080=톰캣.

4. 웹 서버

Nginx·Apache. 정적 파일 + 요청 접수 + 리버스 프록시.

5. WAS

Tomcat·WildFly. 자바 코드 실행 + DB 처리 = 동적 응답.

6. 차이

정적은 웹 서버, 동적은 WAS. 앞뒤로 함께 쓴다.

7. 스프링부트

톰캣을 내장해 jar 하나로 실행. 실무엔 앞에 Nginx.

8. 실익

에러 계층 진단·포트 이해·배포·보안·면접에 직결.

미니 용어사전

클라이언트 / 서버

요청하는 쪽 / 요청을 받아 응답하는 쪽. 하드웨어가 아니라 역할.

DNS

도메인 이름(example.com)을 IP 주소로 바꿔 주는 인터넷의 전화번호부.

포트(Port)

한 서버(IP) 안에서 프로그램을 구분하는 번호. 건물 속 사무실 호수.

HTTP / HTTPS

웹에서 요청·응답을 주고받는 약속. HTTPS는 여기에 암호화(TLS)를 더한 것.

웹 서버(Web Server)

정적 콘텐츠 제공과 요청 접수를 맡는 최전선. Nginx·Apache.

WAS

동적 콘텐츠 생성·비즈니스 로직 실행을 맡는 애플리케이션 서버. Tomcat·WildFly.

서블릿(Servlet)

자바로 웹 요청을 처리하는 표준 방식. WAS가 이를 실행한다.

리버스 프록시

요청을 대신 받아 뒤쪽 WAS로 전달·중계하는 앞단 서버 역할.

로드 밸런싱

요청을 여러 대의 서버에 고르게 나눠 과부하를 막는 것.

내장 톰캣

스프링부트가 프로그램 안에 포함해 함께 실행하는 톰캣(WAS).

더 깊이 공부하려면

- 사내 교육: **스프링부트 교육** — “01 스프링부트 이해하기”, “02 프로젝트 구조(요청 흐름 다이어그램)”
- 사내 교육: **데이터베이스 교육** — WAS 뒤에서 데이터를 책임지는 DB의 기초
- MDN Web Docs — HTTP 개요, 상태 코드 레퍼런스 (한국어 지원)
- 공식 문서 — Nginx, Apache HTTP Server, Apache Tomcat, WildFly 각 프로젝트 사이트

🎓 마치며

이제 브라우저에서 Enter를 누를 때 **보이지 않는 곳에서 무슨 일이 벌어지는지** 그림이 그려질 것입니다. 스프링부트가 대신해 주는 일의 **실체**를 아는 개발자와 모르는 개발자의 성장 속도는 다릅니다. 오늘 배운 “요청 한 번의 여정”을, 다음에 코드를 짜거나 에러를 만날 때 꼭 떠올려 보세요.